

UCS503 - Software Engineering

ReplayDB

A Temporal Database Debugging and Time-Travel Replay System

Submitted by:

Harkirat Singh

BE Third Year, Computer Science and Engineering

Submitted to:

Department of Computer Science and Engineering

Prototype Stage Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Scope and Limitations	3
2	Requirement Analysis	4
2.1	Stakeholders and Requirements	4
2.2	Use-Case Diagram	4
3	Methodology and Design	6
3.1	SDLC Model	6
3.2	System Architecture	6
3.3	Class Diagram	7
3.4	Activity Diagram: Incident Investigation	7
3.5	Technology Stack	7
4	Implementation	9
4.1	Immutable Event Ledger	9
4.2	Replay Studio	9
4.3	Transaction Forensics and Counterfactual Branches	9
4.4	Usability Improvements	9
5	Testing and Evaluation	10
5.1	Testing Methodology	10
5.2	Evaluation Scenario and Results	10
6	Conclusion and Future Work	11
6.1	Summary of Work	11
6.2	Key Findings and Future Work	11

Chapter 1

Introduction

1.1 Background and Context

Modern applications change database records through user actions, scheduled jobs, migrations, and service integrations. When a value becomes incorrect, the current database state rarely explains what changed, who changed it, or which related records were affected. Developers commonly combine logs, backup snapshots, and ad-hoc SQL queries to investigate incidents. This is slow and can itself risk modifying production data.

ReplayDB is an educational temporal-debugging platform for PostgreSQL. It captures row-level mutations in an append-only event ledger and exposes them through an interface for forensic transaction inspection, historical replay, field-level comparison, and read-only counterfactual analysis.

1.2 Problem Statement

Given an unexpected value in a relational database, a developer needs to identify the committed transaction that caused it, inspect all affected rows, reconstruct the record at an earlier point in time, and assess the likely state if the faulty transaction had not occurred. Existing logs are often incomplete and conventional database views normally show only the latest state. ReplayDB addresses this gap through an immutable mutation ledger, deterministic reconstruction, and an explainable forensic workflow.

1.3 Project Objectives

- O1. Capture insert, update, and delete mutations with old state, new state, transaction identifier, timestamp, database user, and query context.
- O2. Reconstruct a record or table state at a selected historical event without changing the live database.
- O3. Group related writes by transaction to expose the blast radius of a suspected incident.

- O4. Compare actual state with a counterfactual timeline in which selected transactions are omitted.
- O5. Provide a clear browser workflow and validate it through automated, API, responsive, and end-to-end tests.

1.4 Scope and Limitations

The prototype supports PostgreSQL-oriented change capture and an in-memory demonstration ledger. It is intended for controlled development and educational investigation, not production restoration. A branch is read-only: it explains a possible alternative state but does not execute a rollback. Later transactions may depend on a removed event; consequently, the default comparison is evaluated at the incident moment so the direct consequence is visible.

Chapter 2

Requirement Analysis

2.1 Stakeholders and Requirements

The primary actor is a developer or database administrator investigating incorrect data. A tester uses the controlled incident scenario to verify replay behaviour. A maintainer configures monitored tables and capture triggers.

ID	Requirement
FR1	Record row-level INSERT, UPDATE, and DELETE events in an append-only ledger.
FR2	Display the chronological history of a selected table record.
FR3	Reconstruct state at any captured event and show field-level differences.
FR4	Group events sharing a transaction ID and show affected tables.
FR5	Create a counterfactual branch excluding one or more transaction IDs.
FR6	Search the ledger and manage monitored table capture.

2.2 Use-Case Diagram

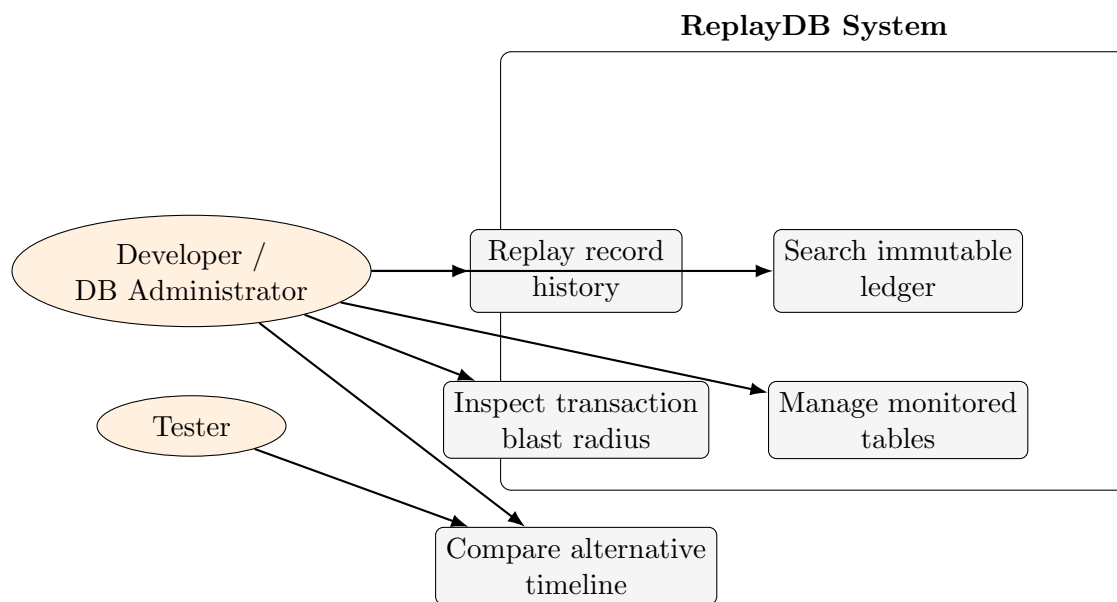


Figure 2.1: Primary use cases for ReplayDB.

Chapter 3

Methodology and Design

3.1 SDLC Model

An iterative and incremental SDLC model was selected. Each iteration delivers a vertical slice: event capture, record replay, transaction forensics, counterfactual comparison, and interface refinement. This approach suits the project because feedback from prototype testing directly informs the next implementation cycle.

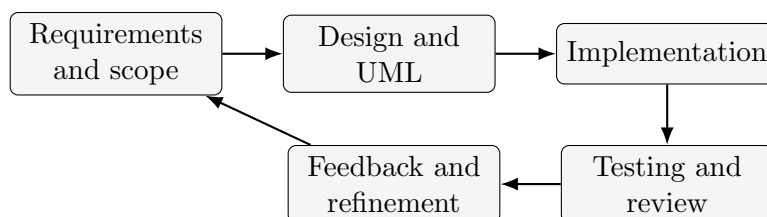


Figure 3.1: Iterative SDLC adopted for the project.

3.2 System Architecture

ReplayDB uses Next.js 15 with TypeScript and React for the user interface and API routes. The API layer exposes ledger, history, replay, transaction, and branch operations. In PostgreSQL mode, PL/pgSQL triggers capture row mutations into `replaydb.replay_events`. In standalone mode, a deterministic in-memory ledger provides the same demonstration workflow without a database installation.

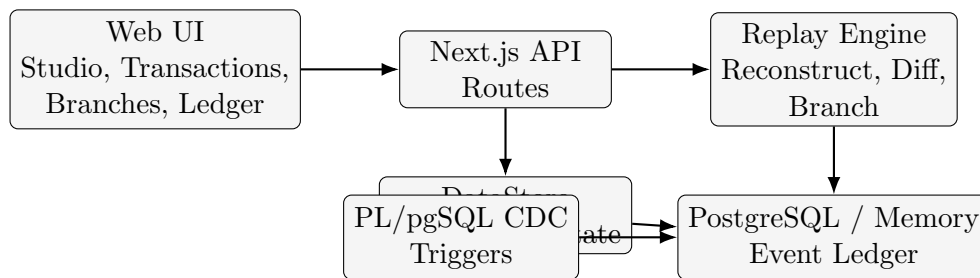


Figure 3.2: High-level ReplayDB architecture.

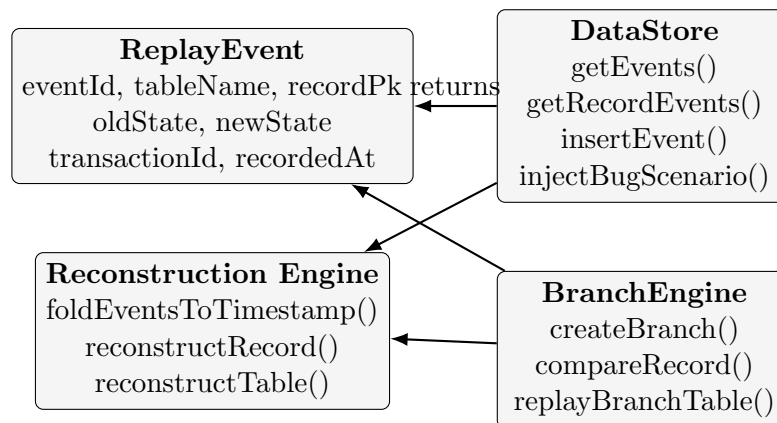


Figure 3.3: Core classes and their responsibilities.

3.3 Class Diagram

3.4 Activity Diagram: Incident Investigation

3.5 Technology Stack

- Front end: Next.js 15, React 19, TypeScript, Tailwind CSS, and Lucide icons.
- Back end: Next.js route handlers and a TypeScript replay engine.
- Database: PostgreSQL-compatible event capture using PL/pgSQL triggers; deterministic memory mode for demos.
- Testing: Vitest unit and integration tests, API verification, responsive visual inspection, and end-to-end scenario testing.

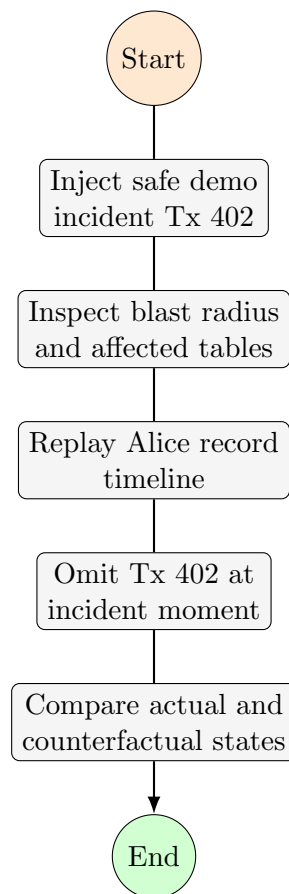


Figure 3.4: Activity flow for the guided incident investigation.

Chapter 4

Implementation

4.1 Immutable Event Ledger

Each mutation is represented as a `ReplayEvent`. It stores the record identity, operation type, old and new JSON states, transaction ID, timestamp, database user, and query context. These fields provide the evidence needed to explain a data change rather than merely display a final value.

4.2 Replay Studio

Replay Studio is the main investigative workspace. The user selects a table and record primary key, views the ordered event timeline, scrubs to a historical point, and reads the reconstructed row state. The field-diff tab highlights attributes changed by the selected mutation.

4.3 Transaction Forensics and Counterfactual Branches

The transaction page groups all events sharing a transaction ID. In the demo, Tx 402 changes the `users` and `accounts` tables, making the cross-table side effect visible as one atomic blast radius. A branch stores a name, base time, and excluded transaction IDs. The branch engine reconstructs the selected record with and without the excluded transaction and lists recovered fields side-by-side.

4.4 Usability Improvements

Visual testing led to a mobile navigation menu, a guided three-step incident workflow, clear error and retry feedback, and incident-moment branch comparison. These changes make the purpose of each page understandable to a first-time user.

Chapter 5

Testing and Evaluation

5.1 Testing Methodology

Testing was performed at four levels: unit tests for reconstruction and diff algorithms, integration tests for API routes and feature interactions, manual API scenario checks, and browser-based end-to-end visual testing at desktop and mobile widths.

Test case	Expected result	Status
Unit and integration suite	Core algorithms and API behaviours remain correct	31/31 passed
Inject Tx 402	Transaction is stored with two affected operations	Passed
Transaction forensics	Users and accounts mutations are shown together	Passed
Counterfactual branch	Tx 402 omitted: FRAUD/-5000 becomes ACTIVE/950.01	Passed
Responsive navigation	Every product page remains reachable on mobile	Passed
Baseline reset	Demo returns to 7 events across 4 monitored tables	Passed

5.2 Evaluation Scenario and Results

The main scenario begins with Alice’s account active with a balance of 950.01. A simulated rogue batch transaction, Tx 402, sets the balance to -5000 and changes the status to FRAUD. Transaction Forensics identifies two affected rows: the user record and the related account record, which is frozen. Branch comparison omits Tx 402 at its incident time and recovers Alice’s active status and original balance. The project therefore links an incorrect record to its causal transaction and demonstrates a read-only alternative state.

Chapter 6

Conclusion and Future Work

6.1 Summary of Work

ReplayDB delivers a working temporal database debugging workflow. It combines change capture, event history, deterministic replay, field-level differences, transaction correlation, and counterfactual comparison in one browser interface.

6.2 Key Findings and Future Work

A usable forensic tool must make its state clear. Showing a raw log is insufficient; the selected record, time, transaction, affected tables, and comparison outcome must be connected in one flow. Future extensions include persistent PostgreSQL deployment, access control, pagination for large ledgers, richer correlation analysis, asynchronous replay, schema-evolution support, and exportable forensic reports.

Bibliography

- [1] Vercel, *Next.js Documentation*. Available: <https://nextjs.org/docs>.
- [2] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Available: <https://www.postgresql.org/docs/>.
- [3] TIET UCS503, *Semester Project Guidance*. Available: <https://tiet-ucs503.github.io/semester-project/>.
- [4] Vitest, *Vitest Documentation*. Available: <https://vitest.dev/>.